

Voice Agent: Deep Dive

A Gemini-driven phone appointment scheduler on a real Twilio line

Working document — generated by exhaustive source read

Contents

1	What this project is	2
1.1	What was actually verified, and what was not	2
2	High-level architecture	2
2.1	Why interfaces, not concrete classes	2
3	Package layout	3
4	The call lifecycle: sequence diagram	3
5	The FastAPI layer: app.py	3
5.1	Twilio signature verification	4
5.2	Dependency injection and lazy app construction	5
6	The conversation state machine: conversation.py	5
6.1	Safety nets, checked first	5
6.2	Slot filling	5
6.3	Booking (<code>_handle_book</code>)	6
6.4	Reschedule and cancel	6
7	The calendar store: calendar_store.py	6
7.1	Interval overlap: the conflict-detection algorithm	6
7.2	SQLiteCalendarStore	7
7.3	GoogleCalendarStore — implemented, not verified	7
8	Intent extraction: intent_engine.py	7
8.1	The IntentEngine interface	7
8.2	GeminiIntentEngine: the real production engine	7
8.3	ScriptedIntentEngine: the free test double	8
9	Session and transcript storage: session_store.py	8
10	Twiml generation: twiml.py	8
10.1	The silent-caller bug and its fix	8
11	The outbound test CLI: cli.py	8

12 Configuration: <code>config.py</code>	9
13 Testing and evaluation strategy	9
13.1 The three-tier testing pyramid	9
13.2 The 25 scripted scenarios	9
13.3 Proving the tests are not vacuous	10
13.4 Fixed time, for reproducibility	10
14 Real API cost incurred (as reported by the project)	10
15 Challenges encountered during the build	10
16 What the project's author would do differently	11
17 Glossary	11

1 What this project is

Voice Agent is a phone-in appointment scheduler. A caller dials a real telephone number; **Twilio** (a cloud service that lets software answer and place phone calls, converting speech to text and text to speech) answers the call and asks a web service — this project — what to say and do next, one conversational turn at a time. That web service uses **Gemini 2.5 Flash** (Google’s large language model, accessed through **Vertex AI**, Google Cloud’s managed ML platform) to figure out, from the caller’s free-form spoken sentence, what they want (book, reschedule, or cancel an appointment) and to pull out the specific details (name, phone number, date, time, duration). A local **SQLite** database (a small, file-based relational database that needs no separate server process) is the actual calendar: it stores every appointment and does real conflict detection, so two callers cannot book the same slot.

The project exists to demonstrate a concrete fix for a real small-business problem: clinics, salons, and repair shops lose bookings to missed calls and voicemail. Instead of a rigid, button-press phone tree (“press 1 for booking”), the caller just talks naturally, and an LLM does the slot-filling a human receptionist would do.

1.1 What was actually verified, and what was not

This distinction matters and is preserved faithfully throughout this document, because the project’s own README is explicit about it:

- **Verified end to end, for real:** the app was booted with the real Gemini engine and real SQLite, exposed over the public internet through an SSH tunnel (`localhost.run`), and sent three requests shaped exactly like Twilio’s real webhook calls, each signed with a genuine HMAC-SHA1 signature (a cryptographic checksum keyed with the real Twilio auth token, the same algorithm Twilio itself uses) computed from the real `TWILIO_AUTH_TOKEN`. Signature validation passed on every request. A 3-turn conversation (opening call, a natural-language booking request, a phone number, a confirmation) resulted in Gemini correctly resolving “tomorrow” against the real current date, and a real row being written to `data/voice_agent.db`: `Test Caller, +15551234567, 2026-07-08T14:00, 30 minutes, status booked` — confirmed by querying the database directly, not by trusting the TwiML response alone. The test row was deleted afterward and the tunnel closed.
- **Never done:** an actual human voice call was never placed. Every verification above used hand-typed English text standing in for what Twilio’s speech-to-text would produce from a real spoken voice. This is the one honestly stated gap: Twilio’s real speech recognition is noisier (mumbling, background noise, partial phrases) than clean typed text, and the Gemini intent engine has never been exercised against that noise.

2 High-level architecture

The whole system is a chain of narrow, swappable pieces. Figure 1 shows how a call flows through them.

2.1 Why interfaces, not concrete classes

An **interface** here means an **abstract base class** (Python’s `abc` module): a class that declares method signatures (`book`, `cancel`, `parse_turn`, etc.) but provides no implementation, forcing every

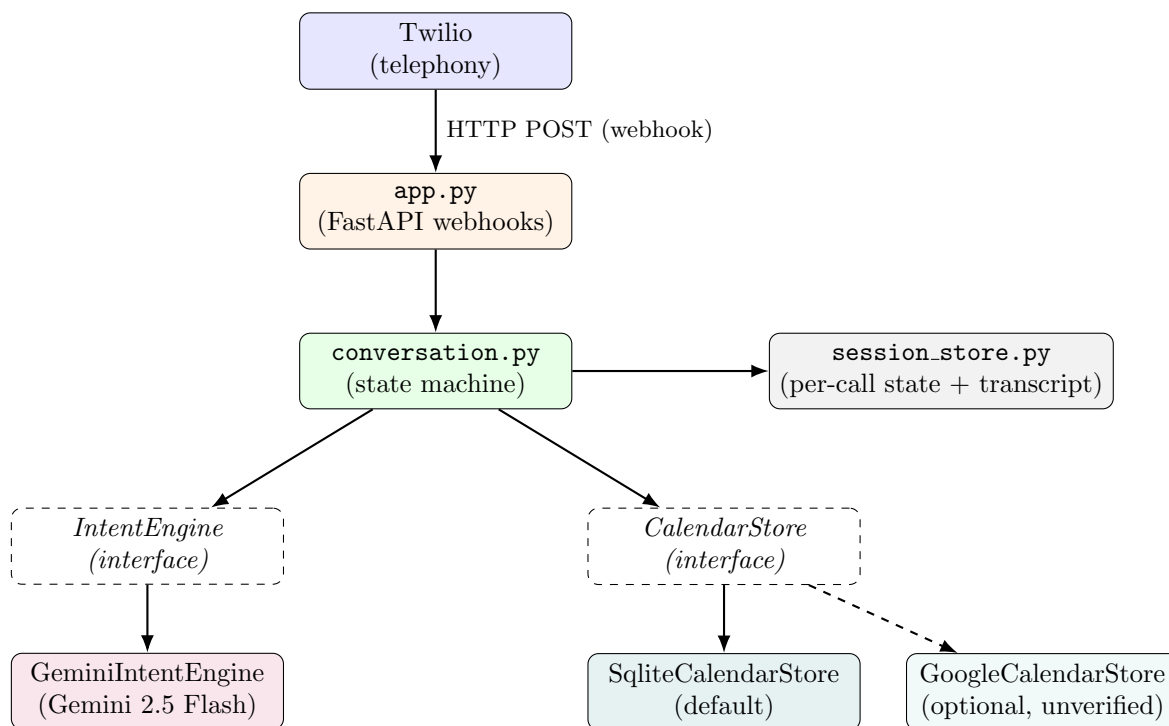


Figure 1: Component map. Dashed boxes are abstract interfaces; solid boxes are concrete implementations. `conversation.py` never imports Gemini or SQLite directly — only the two interfaces — which is the single design decision that makes the rest of this document’s testing story possible.

concrete subclass to fill them in. `IntentEngine` and `CalendarStore` are both such abstract classes. `ConversationManager` (the state machine) is written entirely against these two interfaces and never imports `google.genai` or `sqlite3` directly.

This is what makes it affordable to test an LLM-driven system: swapping in a free, deterministic `ScriptedIntentEngine` (a regex-based stand-in, described in Section 8.3) lets the entire booking/reschedule/cancel logic run through 25 multi-turn scenarios and most of the pytest suite without spending a cent on the real, billed Gemini API. Only a small, explicitly marked slice of tests (Section 13) touches the real API.

3 Package layout

4 The call lifecycle: sequence diagram

Figure 3 traces one webhook round trip in detail: the caller says one sentence, Twilio posts it to the service, and the service replies with new **Twiml** (Twilio Markup Language, an XML dialect that tells Twilio what to say and do next — e.g. speak a sentence, then listen for the next utterance).

5 The FastAPI layer: `app.py`

FastAPI is a Python web framework (a library that turns incoming HTTP requests into Python function calls) used here to expose four HTTP endpoints:



Figure 2: Directory layout. Every module under `voice_agent/` has exactly one job; `eval/` exists purely so the conversation logic can be exercised thousands of times without ever calling the real, billed Gemini API.

- GET `/health` — liveness check, returns `{"status":"ok"}`.
- POST `/voice/incoming` — Twilio calls this the instant a call connects. It reads the caller's number and `CallSid` (Twilio's unique identifier for one call) from the form body, creates or loads session state, logs the greeting to the transcript, and returns TwiML that speaks the greeting and opens a `<Gather>` (a TwiML verb that plays audio, then listens for spoken input).
- POST `/voice/gather` — Twilio calls this every time it captures speech (or a silence timeout, see Section 10). It hands the transcribed text to `ConversationManager.handle_turn`, logs both sides of the exchange, and returns either another `<Gather>` (conversation continues) or a `<Hangup>` (conversation ended).
- POST `/voice/status` — Twilio's call-status callback, fired when the call's state changes (e.g. `completed`, `no-answer`). Used to mark a session `abandoned` if the call ended with no resolved outcome.

5.1 Twilio signature verification

Every incoming webhook can, optionally, be checked against Twilio's `X-Twilio-Signature` header. This header is an **HMAC-SHA1** signature — a value computed by feeding the exact request URL

and form body through a keyed hash function (HMAC: Hash-based Message Authentication Code; SHA-1: the specific hash algorithm) using the account's secret `TWILIO_AUTH_TOKEN` as the key. If an attacker does not know that token, they cannot produce a signature that matches, so verifying it proves the request really came from Twilio (or from anyone who also knows the token) and was not tampered with in transit. The check (`verify_twilio_signature`) only actually runs when both `PUBLIC_WEBHOOK_BASE_URL` and `TWILIO_AUTH_TOKEN` are configured, since Twilio signs the exact public URL it fetched, and the server needs that same URL string to recompute the same signature.

5.2 Dependency injection and lazy app construction

`create_app(...)` is a factory function taking optional `calendar_store`, `session_store`, `intent_engine`, and `enforce_signature` arguments: called with no arguments (as `uvicorn` does in production) it wires up the real `GeminiIntentEngine` and the real SQLite stores; called with explicit stores (as every test does) it swaps in fakes/temp databases instead. This pattern — passing dependencies in rather than hard-coding them — is called **dependency injection**.

The module-level `app` object (what `uvicorn voice_agent.app:app` imports) is built lazily via Python's `module __getattr__` hook (PEP 562): merely importing `app.py` never touches real credentials; only actually accessing the `app` attribute builds the real production app. This lets every test import the module freely without needing a live Gemini/Twilio credential present.

6 The conversation state machine: `conversation.py`

This is the business-logic core. `ConversationManager.handle_turn(session, utterance)` is called once per caller utterance and returns a `TurnOutcome` (a reply string, whether to end the call, and an optional terminal outcome label).

6.1 Safety nets, checked first

1. **Turn cap:** if `session.turn_count` exceeds `MAX_TURN_COUNT` (14), the call is forced to `handoff` regardless of what is happening, closing off any conversation that has gone on unreasonably long (e.g. a stuck loop).
2. **Intent-engine failure:** if `parse_turn` raises `IntentEngineError` (any Gemini API failure — network error, quota, malformed output), the call ends immediately with a clear technical-failure message rather than leaving the caller stuck in silence.
3. **Unclear speech / retry cap:** if the model's intent is "unclear", a `retry_count` increments; after `MAX_RETRY_COUNT` (3) consecutive unclear turns, the call hands off instead of looping forever.

6.2 Slot filling

A **slot** here is one piece of information the agent needs before it can act: `name`, `phone`, `date_iso`, `time_24h`, `duration_minutes`, plus a `confirmed` yes/no flag. Each turn, the model (or the scripted stand-in) returns whatever slots it could extract from the *entire* conversation so far, and any newly extracted, non-null value overwrites what was previously known. If the caller has no phone slot filled in yet, the caller's own number (from Twilio's `From` field) is used as a sensible default.

If the caller's newly detected intent differs from the action already in progress (e.g. they were booking, then say “actually, cancel my other appointment”), the manager treats it as a clean restart: the new action replaces the old one and all slots are cleared, rather than mixing data from two different intents.

6.3 Booking (`_handle_book`)

1. If `name`, `phone`, `date_iso`, or `time_24h` is still missing, just relay the model's own reply (which is asking for the next missing piece).
2. Once all four are known, nothing books until `confirmed` is `True` — an explicit yes from the caller to a read-back summary. This is a deliberate human-in-the-loop check before writing to the calendar.
3. The combined date+time string is parsed into a real `datetime` object with `_parse_slot_datetime`; if parsing fails, the time slot and confirmation are cleared and the caller is asked again.
4. `_validate_business_slot` rejects times in the past or outside business hours (9:00–17:00 by default, see `config.py`), resetting the time/confirmation so the caller can try a different time.
5. `calendar_store.book(...)` is called with the duration (or the 30-minute default). If it raises `ConflictError` (the slot overlaps an existing booked appointment), the caller is told which time is taken and asked for another.
6. On success, the session's outcome becomes “booked”, the action is cleared, and a natural-language confirmation reply is built directly from the saved `Appointment` object (not from the model's own text), guaranteeing the spoken confirmation matches exactly what was actually written to the database.

6.4 Reschedule and cancel

Both first need to locate the caller's existing appointment (`_find_target_appointment`, which looks up the caller's phone number in the calendar and takes the first active match) if one is not already targeted this call. If none is found, the caller is offered to book a new appointment instead. **Cancel** then walks through an explicit confirm/reject exchange before actually calling `calendar_store.cancel`. **Reschedule** needs a new date and time, runs the exact same business-hours and conflict checks as booking (excluding the appointment's own current slot from the conflict check, so moving an appointment doesn't collide with itself), then calls `calendar_store.reschedule`.

7 The calendar store: `calendar_store.py`

7.1 Interval overlap: the conflict-detection algorithm

Two appointments conflict if their time ranges overlap. The code uses a **half-open interval** check:

```
def _overlaps(a_start, a_end, b_start, b_end) -> bool:
    return a_start < b_end and b_start < a_end
```

A half-open interval treats the start as included and the end as excluded (`[start, end)`). The practical effect: an appointment ending at 10:00 does *not* conflict with one starting at 10:00 — they touch but do not overlap — which matches how a real receptionist would schedule back-to-back appointments. This is verified by an explicit boundary test in `tests/test_calendar_store.py`.

7.2 SqliteCalendarStore

The default and only backend actually exercised by this project’s own tests. One table, `appointments`, with columns `id`, `name`, `phone`, `start_time`, `duration_minutes`, `status` ("booked"/"cancelled"), and timestamps. `find_conflict` scans every currently-booked row and checks overlap in Python (not in SQL) — fine at the scale of one phone line’s appointment volume, but something a busier deployment would want to push into an indexed SQL range query.

7.3 GoogleCalendarStore — implemented, not verified

A second implementation of the same `CalendarStore` interface, talking to the real Google Calendar API v3 (`events.insert/patch/delete`, plus `freebusy.query` for conflict detection). It is selected automatically by the `get_calendar_store()` factory only when `GOOGLE_CALENDAR_CREDENTIALS_PATH` is set and points at a real file. The README and the code comments are explicit: this adapter is fully implemented (not a stub) but **no Google Calendar credentials were ever provided**, so it has never actually run against a live calendar. This is a stated, honest gap, not a hidden one.

8 Intent extraction: `intent_engine.py`

8.1 The IntentEngine interface

`parse_turn(ctx: TurnContext) -> TurnResult` is the entire contract. A `TurnContext` bundles everything the model needs for one turn: the caller’s phone number, their latest utterance, the last 12 transcript turns (for conversational memory), whatever slots are already known, the current action (if one has been established), and today’s date (so relative dates like “tomorrow” resolve correctly).

8.2 GeminiIntentEngine: the real production engine

Talks to Gemini 2.5 Flash through Vertex AI using the `google-genai` Python SDK. Three configuration choices matter and are all deliberate, motivated by one constraint: a caller on a live phone call cannot tolerate multi-second silences.

- **Thinking disabled** (`ThinkingConfig(thinking_budget=0)`). Gemini 2.5 Flash supports an internal “thinking” mode that spends extra tokens reasoning before answering; turning it off is a measured decision — the README records an early exploratory call with default settings that took noticeably longer and spent 56 thinking tokens on a one-word answer, versus 1.8 seconds with thinking disabled.
- **Strict JSON schema** (`response_schema + response_mime_type="application/json"`). The model is constrained to return exactly the fields the code expects (`intent`, `reply`, `name`, `phone`, `date_iso`, `time_24h`, `duration_minutes`, `confirmed`), rather than free text the code would have to parse with more fragile heuristics.
- **Low temperature** (0.1). **Temperature** controls how much randomness the model injects into its output; a low value makes responses more consistent and deterministic run to run, appropriate for a structured extraction task rather than creative writing.

The system prompt (`SYSTEM_PROMPT_TEMPLATE`) tells the model today’s date, business hours, the default appointment duration, and exactly what JSON shape to produce; it is re-rendered fresh every turn with the current known slots and action baked in, so the model always has full up-to-date

context even though each API call is independently stateless. If the API call fails for any reason (network, quota, auth), `IntentEngineError` is raised and propagates up to `ConversationManager`, which turns it into an immediate, graceful hangup message rather than an unhandled exception.

8.3 ScriptedIntentEngine: the free test double

Lives in `eval/scripted_engine.py`, not in the production package, and its own docstring is explicit that it is a **test double** (a stand-in object used only in tests, never in production) rather than a production fallback. It is a regex/keyword matcher: pattern-matches phrases like “book”, “reschedule”, “cancel”, “yes”/“no”, extracts names via patterns like “my name is X”, phone numbers via a digit-sequence regex, and dates/times via weekday names, month names, and numeric formats. It is intentionally simple — only as good as the phrases the 25 scenarios and tests actually use — because its only job is exercising the real conversation state machine and calendar store for free, not approximating real natural-language understanding.

9 Session and transcript storage: `session_store.py`

Twilio’s `<Gather>` pattern means each spoken exchange is a completely separate, stateless HTTP request — nothing survives in server memory between turns unless explicitly persisted. `SessionStore` solves this with a second SQLite database (separate from the calendar), keyed by `CallSid`: a `sessions` table holding the entire `SessionState` dataclass serialized to JSON, and a `transcript` table logging every line either side said, in order, for later auditing. `mark_abandoned_if_unresolved` is called from the `/voice/status` webhook: if a call ends (completed, failed, busy, no-answer, canceled) and its session never reached a terminal outcome, it is retroactively marked “abandoned”.

10 TwiML generation: `twiml.py`

Kept deliberately separate from `app.py` so it can be unit tested without booting FastAPI at all. `build_gather_twiml` builds a `<Gather>` block: speak some text, then listen for speech for up to 6 seconds (`GATHER_TIMEOUT_SECONDS`), with `speech_timeout="auto"` (Twilio decides when the caller has finished speaking).

10.1 The silent-caller bug and its fix

By default, Twilio’s `<Gather>` only re-requests its `action` URL when it actually captures speech; on a silent timeout, it just falls through to whatever TwiML comes next in the same response, bypassing the server entirely. That would have made the retry-count and handoff logic in `conversation.py` completely unreachable for a caller who simply says nothing. The fix, documented as a real challenge encountered during the build, is `action_on_empty_result="true"`: every outcome, whether real speech or silence, now posts back to the same `/voice/gather` endpoint, so one code path in the conversation manager handles both cases uniformly.

11 The outbound test CLI: `cli.py`

A human-operated command-line tool, never invoked automatically by anything else in the codebase, that places a real, billed outbound Twilio call to a phone number passed explicitly as a command-line argument. It validates the number looks like E.164 format (the international phone

number standard, e.g. +15551234567), refuses to run without a configured public webhook URL (since Twilio needs a URL it can fetch TwiML from over the internet), and requires an explicit `y` confirmation (or `--yes`) before dialing, specifically because it costs real money and rings a real phone.

12 Configuration: `config.py`

Centralizes every environment variable read (Twilio credentials, GCP project, webhook base URL, database path) and the business-rule constants: business hours 9:00–17:00, default appointment duration 30 minutes, max 3 consecutive unclear turns before handoff, max 14 total turns before a forced handoff. Values are read from `os.environ` at call time, never cached at import time, so tests can freely override them per-test via environment variables or explicit constructor arguments.

13 Testing and evaluation strategy

13.1 The three-tier testing pyramid

Layer		What it exercises	Real, billed dependency?
Unit	tests	Booking, conflicts, reschedule, cancel, boundary overlap (<code>test_calendar_store.py</code>)	No — temp SQLite only
Endpoint	tests	FastAPI TestClient, full multi-turn flow through real HTTP handlers (<code>test_twiml_endpoints.py</code>)	No — scripted engine, temp SQLite
25 scripted scenarios		The real conversation state machine and real temp run via SQLite, every action and edge case (<code>eval/scenarios.py</code> , <code>test_eval_scenarios.py</code>)	No — scripted engine
		The actual <code>GeminiIntentEngine</code> against the live Vertex AI credential (<code>test_llm_intent.py</code> , marked <code>real_api</code>)	Yes — 6 real Gemini calls
		Webhook signature HMAC check (no network) plus one read-only Twilio phone-number lookup (<code>test_twilio_integration.py</code> , marked <code>real_api</code>)	Partly — one free, read-only REST call

The full suite is **54 tests, 54 passed, 0 failed** (47 run by default with `pytest -m "not real_api"`; the other 7 run separately, by design, so real API spend stays visible and deliberate rather than happening on every test run).

13.2 The 25 scripted scenarios

Cover: one-shot and multi-turn booking, explicit duration, explicit phone override, rejected-then-corrected confirmation, reschedule (success, not-found, conflict-then-alternate), cancel (success, not-found), double-booking and overlapping/adjacent conflicts, ambiguous time and date, unclear speech (both recovering and exceeding the retry cap into handoff), an abandoned call detected

via the status callback, business-hours and past-date rejection, a mid-call change of mind, two independent concurrent bookings, a goodbye-after-success not clobbering the recorded outcome, and the absolute turn-count safety cap.

13.3 Proving the tests are not vacuous

A genuinely notable piece of engineering discipline recorded in the README: the retry cap and turn cap were deliberately, temporarily broken in a throwaway run to confirm the two scenarios that check them actually *fail* when the invariant is broken, rather than passing regardless of whether the code does anything at all. Both failed as expected; the sabotage was then reverted. This is exactly the check that distinguishes a real regression test from a test that merely never triggers its own assertion.

13.4 Fixed time, for reproducibility

Every scripted scenario runs against a fixed reference “now” (`FIXED_NOW = datetime(2026, 7, 6, 13, 0)`) instead of the real wall clock, threaded through `ConversationManager` via an injectable `now_fn` parameter. Without this, a scenario asserting an exact resulting timestamp (e.g. resolving “tomorrow”) would only be correct on the one calendar day it happened to run, and business-hours/past-date checks would silently drift as time passes.

14 Real API cost incurred (as reported by the project)

Gemini: 12 real `generateContent` calls total across the entire build and verification history (2 early exploratory calls, 6 in the marked test file, 1 instrumented latency measurement, plus 3 more from the live public-tunnel verification, one per conversational turn). At roughly \$0.30 per million input tokens and \$2.50 per million output tokens for Gemini 2.5 Flash, the total across all 12 calls is estimated at under a tenth of a cent (a rate-based estimate, not a verified line-item invoice). Twilio: three real, read-only REST API calls total (an account phone-number lookup, a webhook-config fetch during tunnel verification, and a phone-number SID lookup); Twilio does not bill for read-only calls, and no call, SMS, or paid Lookup API was ever used, so the real Twilio cost incurred is \$0.00.

15 Challenges encountered during the build

1. **Silent callers bypassing the retry logic**, fixed with `action_on_empty_result="true"` (Section 10).
2. **Testing an LLM-driven system on a real budget**, solved by the `IntentEngine` interface plus the `ScriptedIntentEngine` test double (Section 8.3).
3. **Non-deterministic relative dates breaking test reproducibility**, fixed by threading an injectable `now_fn` through the conversation manager and pinning every scenario to `FIXED_NOW`.
4. **Scenarios that looked correct but didn’t exercise the intended code path**: an early draft of the business-hours/past-date rejection scenarios let the caller correct a bad time before ever confirming it, so the validation function (which only runs after confirmation) never actually ran. Caught by tracing exact turn-by-turn state transitions, fixed by adding an explicit confirming “yes” turn first.

5. **Gemini’s default thinking mode being too slow for a live phone turn**, addressed by disabling thinking and constraining output with a strict JSON schema (Section on **GeminiIntentEngine**).
6. **FastAPI’s url_for returning an absolute URL**, not the bare relative path a first draft test assumed — caught by an actual failing pytest run, fixed in the test’s assertion, not the application code (the absolute URL is the real, correct fallback behavior).

16 What the project’s author would do differently

- **No timezone handling at all**: every date/time is treated as one implicit local timezone for both the business and every caller. A real multi-region deployment would need to ask for or infer the caller’s timezone.
- **The real Gemini engine is only proven against clean, hand-typed English**, never against actual noisy Twilio speech-to-text output — the single honestly stated gap between “the API call works” and “this works on a real phone call.”
- **The Google Calendar adapter is fully implemented but never run against a live calendar**, since no credentials were provided.
- **SQLite is single-writer**: fine for one phone line, but a deployment with multiple concurrent lines or worker processes would need Postgres (or WAL mode plus real concurrency testing).
- **An actual human voice call was never placed**: the closest approximation is the public-tunnel HMAC-signed text verification described in Section 1.1, still short of a live phone in the loop.

17 Glossary

Twilio

A cloud telephony platform: lets software programmatically answer, place, and control phone calls, and converts speech to text and text to speech.

Twiml

Twilio Markup Language: an XML dialect the server returns to tell Twilio what to say and do next (speak, listen, hang up).

Vertex AI

Google Cloud’s managed platform for calling Google’s ML models, including Gemini, with enterprise credentials and quotas.

FastAPI

A Python web framework for building HTTP APIs, used here to expose the four webhook endpoints Twilio calls.

SQLite

A small, file-based relational database requiring no separate server process; used here for both the calendar and per-call session state.

Abstract base class / interface

A class that declares method signatures without implementing them, forcing subclasses to

provide the implementation; used here so business logic can depend on a contract rather than a concrete vendor.

Test double

A stand-in object used only in tests (here, `ScriptedIntentEngine`), never in production.

HMAC-SHA1

A cryptographic signature computed by hashing data together with a secret key (HMAC) using the SHA-1 hash algorithm; used to verify Twilio webhook requests are authentic.

Half-open interval

A range that includes its start but excludes its end (`[start, end)`); used for appointment overlap so back-to-back appointments don't falsely conflict.

E.164

The international standard format for phone numbers, e.g. `+15551234567`.

Dependency injection

Passing an object's dependencies in from outside (e.g. as constructor arguments) instead of constructing them internally, so tests can substitute fakes.

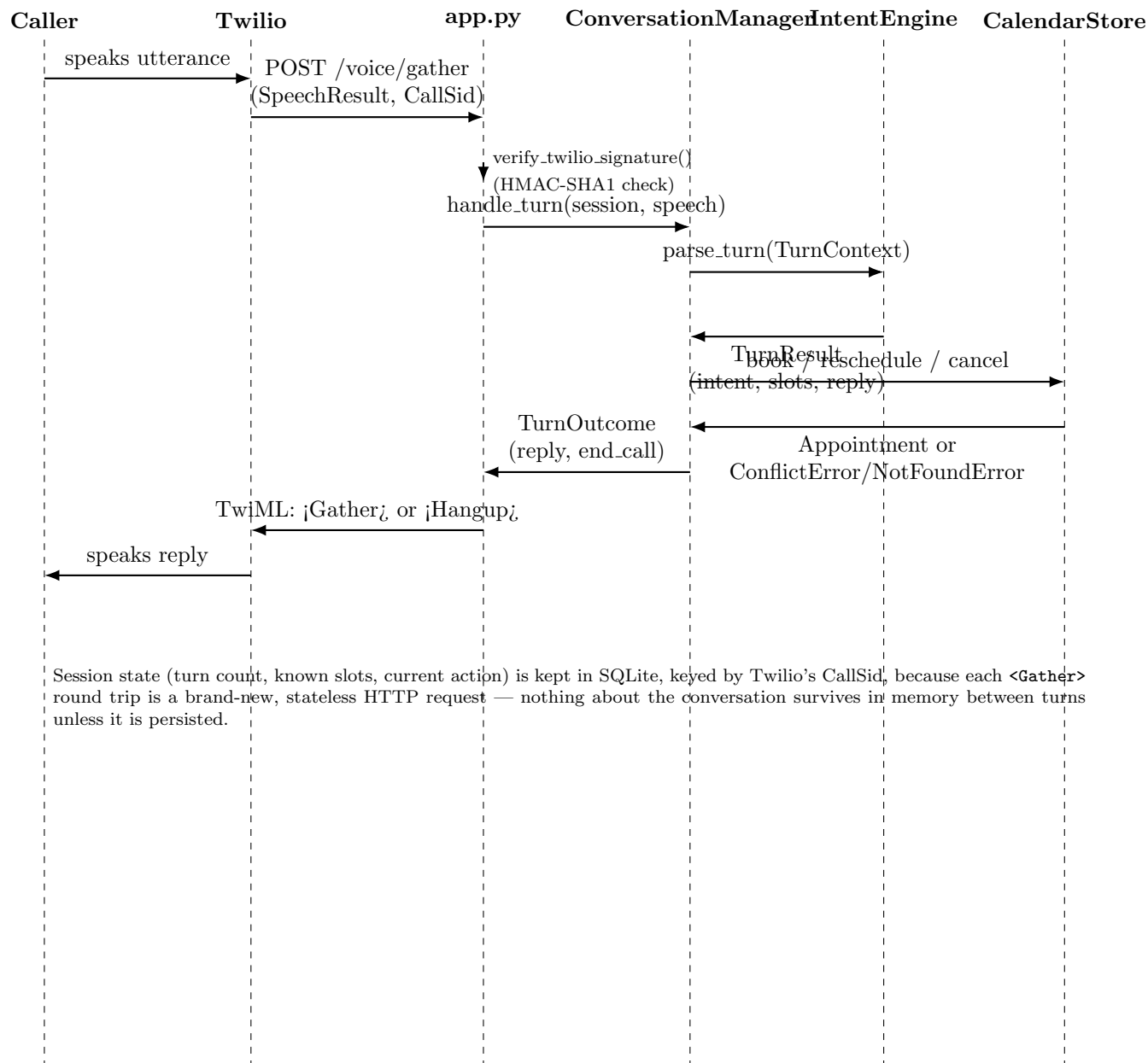


Figure 3: One turn of a live call, end to end. The same shape repeats for every utterance until the call reaches a terminal outcome (booked, rescheduled, cancelled, handoff, or abandoned) and a `<Hangup>` is issued.